

Agustín König - Sistema de Gestión de Seguros - Segunda Entrega

1. Ajustes de la Entrega N°1

Luego de la primera entrega, se aplicaron las siguientes mejoras de normalización e integridad:

- Se eliminó la restricción de unicidad total sobre el DNI en la tabla Asegurados para contemplar excepciones, delegando la identificación unívoca a la clave primaria *id_asegurado*.
- **Normalización de los pagos:** Creé la tabla dimensional Metodos_Pago, transformando el campo de texto en una clave foránea (FK), para reducir la redundancia y asegurar la consistencia de los datos.
- Implementé acciones de DELETE CASCADE en las relaciones clave, garantizando que no existan datos huérfanos, como pagos o siniestros sin una póliza válida.

2. Listado de Vistas (3)

- **Vista_Polizas_Activas:**

Objetivo: Facilitar la gestión comercial diaria identificando contratos vigentes.

Tablas que la componen: Asegurados, Polizas.

Descripción: Presenta de forma homogénea los nombres de los clientes junto a sus números de póliza y montos asegurados, filtrando solo aquellos con estado 'Activa'.

- **Vista_Analisis_Financiero:**

Objetivo: Controlar el flujo de caja por contrato para reportes de rentabilidad.

Tablas que la componen: Polizas, Pagos.

Descripción: Consolida la información financiera invocando la función total_pagado para mostrar cuánto se ha recaudado por cada póliza.

- **Vista_Siniestralidad_Cliente:**

Objetivo: Evaluar el riesgo de cada asegurado basándose en su historial de incidentes.

Tablas que la componen: Asegurados, Polizas, Siniestros.

Descripción: Une las tres tablas para listar los siniestros (fecha y monto reclamado) asociados a cada nombre de cliente.

3. Funciones Personalizadas (2)

- **total_pagado (poliza INT):**

Objetivo: Obtener el balance acumulado de pagos de un contrato.

Datos/Tablas: Manipula la tabla Pagos utilizando la función integrada SUM().

Descripción: Recibe el ID de una póliza y devuelve la suma total del campo monto, manejando valores nulos con IFNULL.

- **cantidad_siniestros (poliza INT):**

Objetivo: Cuantificar el uso de la cobertura por parte del asegurado.

Datos/Tablas: Consulta la tabla Siniestros.

Descripción: Retorna el total numérico de registros asociados a una póliza mediante el comando COUNT(*).

4. Stored Procedures (2)

- **registrar_pago:**

Beneficio: Estandariza la carga de datos financieros evitando errores de inserción manual.

Interacción: Inserta registros en la tabla Pagos recibiendo parámetros de monto, método y póliza.

- **registrar_siniestro:**

Beneficio: Automatiza el reporte de incidentes centralizando la descripción y montos reclamados.

Interacción: Realiza una operación INSERT en la tabla Siniestros tras recibir datos dinámicos como parámetros de entrada.

5. Triggers (2)

- **validar_pago:**

Funcionalidad: Actúa como una restricción de integridad de negocio preventivo.

Situación: Se dispara BEFORE INSERT en la tabla Pagos.

Descripción: Evalúa el nuevo valor (NEW.monto) y bloquea la operación si el monto es menor o igual a cero, garantizando la calidad de los datos.

- **actualizar_estado_poliza:**

Funcionalidad: Automatización de flujos de estado tras un evento crítico.

Situación: Se dispara AFTER INSERT en la tabla Siniestros.

Descripción: Tras registrarse un siniestro, ejecuta un UPDATE automático en la tabla Polizas para cambiar el estado a 'Con siniestro'.

6. Inserción de Registros (10)

Para la prueba del sistema, se procedió a poblar la base de datos utilizando el sublenguaje DML, mediante la sentencia **INSERT**. Siguiendo los requerimientos de la entrega, se cargaron 10 registros en cada una de las tablas principales, lo que permite realizar pruebas de funcionalidad y verificar que los objetos programables, vistas, funciones y procedimientos, devuelvan los resultados esperados.

El proceso de inserción siguió un orden lógico basado en la jerarquía de las relaciones. Se poblaron primero las tablas primarias (Asegurados, Tipos_Seguro y Metodos_Pago) antes de las tablas secundarias o transaccionales (Polizas, Pagos, Siniestros).

Este enfoque garantiza que cada clave foránea en las tablas secundarias apunte a una clave primaria válida en las tablas principales, evitando la existencia de "datos huérfanos" y asegurando que las relaciones entre tablas permanezcan precisas y sincronizadas.